C++ Projectopdracht: Winkelkassa

Overzicht

Ontwerp en implementeer een inventarisatie- en kassasysteem voor een klein buurtwinkeltje. De winkel verkoopt verschillende soorten producten (voeding en non-food), elk met hun eigen prijsberekening en BTW-tarief. Daarnaast kunnen klanten hun aankopen meenemen in verschillende types tassen, elk met een eigen capaciteit en kostprijs.

Je taak is om dit systeem te modelleren in modern C++ met behulp van klassen, polymorfisme, smart pointers, STL-containers, templates en foutafhandeling.

In deze opdracht worden er interfaces vastgelegd waar je je aan moet houden. Je mag zelf extra methodes, klassen of dataleden toevoegen als je denkt dat dit nodig is, maar de gegeven interfaces moeten correct geïmplementeerd worden. Als een klasse een pure virtuele functie heeft, maar je denkt dat deze functie ook in de basisklasse geïmplementeerd kan worden, dan mag je dit uiteraard doen. Het belangrijkste is dat de gegeven interfaces correct geïmplementeerd worden en dat het systeem werkt zoals beschreven. Dit wil dus zeggen dat je een pure virtuele functie mag omzetten naar een virtuele functie met een implementatie in de basisklasse. Je mag zelfs een virtuele functie niet-virtueel maken als je denkt dat dit beter is voor je ontwerp. Echter als een functie in een bepaalde klasse staat **moet** deze functie beschikbaar zijn in deze klasse. Dus stel dat ik volgende interface voorstel:

```
class Product
{
public:
    virtual print_info() const = 0;
};
```

Dan moet de functie `print_info` altijd beschikbaar zijn in de klasse `Product`. Wat je mag doen is deze functie implementeren in de klasse `Product` zelf, zodat deze niet langer een pure virtuele functie is, maar nog steeds virtueel. Of je kan deze functie zelfs helemaal niet-virtueel maken als je denkt dat dit beter is voor je ontwerp.

```
class Product
{
public:
    virtual void print_info() const; // nu niet langer pure virtueel, maar nog steeds virtueel
};
```

of

```
class Product
{
public:
    void print_info() const; // nu niet langer virtueel, maar nog steeds beschikbaar in Product
};
```

Ook de zichtbaarheid van dataleden en methodes moeten hetzelfde blijven voor de interface die gegeven is. In het voorbeeld hierboven moet `print_info` steeds publiek blijven, ook al zou je deze functie liever privé maken in je implementatie.

Dus kort samengevat wat je **wel** mag doen:

- Extra methodes, klassen of dataleden toevoegen
- Pure virtuele functies implementeren in de basisklasse (omzetten naar virtuele functies met implementatie)
- Virtuele functies niet-virtueel maken

Wat je **niet** mag doen:

- Verwijderen van methodes die in de gegeven interfaces staan
- Veranderen van de zichtbaarheid van methodes of dataleden die in de gegeven interfaces staan
- Veranderen van de naam van methodes die in de gegeven interfaces staan
- Veranderen van de parameters (type en aantal, namen mag je aanpassen) of return types van methodes die in de gegeven interfaces staan
- Veranderen van de gegeven klassenhiërarchie (dus als er staat dat een klasse van een andere klasse erft, dan moet dit ook zo blijven)

Als je toch denkt te moeten afwijken van de gegeven interfaces, dan kan je dit altijd bespreken met de docent of assistent. Het belangrijkste is dat je een goed ontwerp maakt dat werkt zoals beschreven in de opdracht, en dat je de gegeven interfaces correct implementeert.

Leerdoelen

- Ontwerpen van klassenhiërarchieën met overerving
- Gebruik van virtuele functies en runtime polymorfisme
- Gebruik van `std::vector`, `std::unordered_map`, `std::unique_ptr`
- Begrijpen van abstracte basisklassen en interface-ontwerp
- Gebruik van templates en `std::derived_from`
- Inventarisbeheer en foutafhandeling
- Toepassen van RAII en moderne C++-stijl

---

1. Producthiërarchie

De winkel verkoopt *food* (6% btw) en *non-food* (21% btw) producten. Alle producten erven af van de abstracte klasse `Product`.

1.1 Abstracte klasse `Product`

```
class Product
{
public:
    virtual ~Product() = default;

    [[nodiscard]] virtual std::string const& name() const = 0;
    [[nodiscard]] virtual std::string const& type() const = 0;
    [[nodiscard]] virtual double base_price() const = 0;
    [[nodiscard]] virtual double vat_percentage() const = 0;

    [[nodiscard]] double price() const = 0;
};
```

1.2 Food (6% btw)

```
class Food : public Product
{
public:
};
```

1.3 Non-Food (21% btw)

```
class NonFood : public Product
{
public:
};
```

1.4 Concrete producten

Je moet minstens **vier concrete producten** maken: - Minstens 2 food-producten - Minstens 2 non-food-producten

Voorbeeld (food): Apple

```
class Apple : public Food
{
    double weight_;
    double price_per_kg_;
public:
    explicit Apple(double weight, double price_per_kg);
};
```

Voorbeeld (non-food): Soap

```
class Soap : public NonFood
{
    double price_;
public:
    explicit Soap(double price);
};
```

---

2. Tas-hiërarchie

De klant kiest een tas. Dit gedrag is polymorf.

2.1 Abstracte klasse Bag

```
class Bag
{
public:
    virtual ~Bag() = default;

    [[nodiscard]] virtual std::string const& type() const = 0;
    virtual void add_item(std::unique_ptr<Product> item) = 0;
    [[nodiscard]] virtual std::unordered_map<std::string, int> contents() const = 0;
};
```

2.2 Concrete tas-types (verplicht)

Je implementeert: - **PlasticBag** (goedkoop, medium capaciteit (bv 10 producten)) - **BioBag** (biologisch afbreekbaar, lage capaciteit (bv 5 producten)) - **ReusableBag** (duur, grote capaciteit (bv 25 producten))

Elke tas gebruikt:

```
std::vector<std::unique_ptr<Product>> items_;
```

Elke tas bepaalt zelf: - maximale capaciteit - eventuele kostprijs (optioneel)

---

3. Shop-klasse

De Shop beheert inventaris van producten en tassen.

```
class Shop
{
public:
    void add_product(unique_ptr<Product> product);
    void add_products(std::vector<unique_ptr<Product>> products);

    void add_bag(unique_ptr<Bag> bag);
    void add_bags(std::vector<std::unique_ptr<Bag>> bags);

    [[nodiscard]] std::unordered_map<std::string, int> product_inventory() const;
    [[nodiscard]] std::unordered_map<std::string, int> bag_inventory() const;

    [[nodiscard]] std::unique_ptr<Product> take_product(std::string const& name);
    [[nodiscard]] std::unique_ptr<Bag> take_bag(std::string const& type);

private:
    std::vector<std::unique_ptr<Product>> products_;
    std::vector<std::unique_ptr<Bag>> bags_;
};
```

---

4. Hoofdprogramma (CLI)

4.1 Setup

Je kan items op de volgende manier toevoegen:

```
Shop shop{};
// Voeg producten één per één toe
shop.add_product(std::make_unique<Apple>(0.2, 0.2));
shop.add_product(std::make_unique<Cheese>(0.5, 3.50));
shop.add_product(std::make_unique<Soap>(1.25));
shop.add_product(std::make_unique<Sponge>(0.90));

// Voeg tassen een aantal keer toe
for(int i = 0; i < 5; ++i) {
    shop.add_bag(std::make_unique<PlasticBag>());
    shop.add_bag(std::make_unique<BioBag>());
    shop.add_bag(std::make_unique<ReusableBag>());
}
```

4.2 Interactie

1. Toon beschikbare tassen
2. Vraag de gebruiker een tas te kiezen
3. Toon productinventaris + prijzen inclusief btw

4. Gebruiker kiest product en hoeveelheid/parameters
5. Voeg product toe aan tas
6. Vraag *Nog een product? (y/n)*
7. Druk uiteindelijk de volledige bon af

Voorbeeldoutput

```
Beschikbare tassen:
- plastic (nog 5)
- bio (nog 5)
- reusable (nog 3)
Welke tas wil je? bio

Beschikbare producten:
- apple (€0.42) (nog 12)
- cheese (€3.71) (nog 5)
- soap (€1.51) (nog 8)
- sponge (€1.09) (nog 6)

Welk product wil je? apple
Hoeveel kg? 0.15
Nog een product? (y/n) y
...

--- BON ---
Tas: bio (€0.15)
Producten:
- apple: €0.42
- cheese: €3.71
Totaal: €4.28
```

Foutafhandeling

- Geen tassen meer → meld dit
- Product niet beschikbaar → vraag opnieuw
- Ongeldige invoer → opnieuw vragen
- Tas vol → vraag om andere tas of stoppen

Bonusopdrachten

- sorteren van producten op prijs
- meerdere tassen in één bestelling
- extra promoproducttypes implementeren

Veel succes en veel plezier met het bouwen van je winkelkassa!

Laatste wijziging: donderdag, 2 april 2026, 12:12

◄ Labo 11

Ga naar...

Indienen opdracht winkelkassa ►